เอกสาร SA

1. Microservices Architecture
2. Domain-Driven Design (DDD)
3. Clean Architecture
4. บริหารความเสี่ยงโครงการ
5. วางแผน POC (Proof of Concept)

> **หมายเหตุการจัดเรียง:** แต่ละหัวข้อกล่าวถึงเฉพาะบริบทของตน ไม่ repeats รายการที่ซ้ำ เช่น KPI/ข้อดี/ข้อเสีย จะปรับให้ตรงหัวข้อนั้น และรวมหัวข้อย่อยไว้ใน “สาระสำคัญ” หรือตารางเดียว

---

1. Microservices Architecture

เรื่อง

สถาปัตยกรรมไมโครเซอร์วิส — แบ่งระบบเป็นบริการย่อยอิสระ สื่อสารผ่าน API และ deploy/scale แยกกัน

วัตถุประสงค์

ออกแบบระบบให้แต่ละบริการมีขอบเขตชัด ลดการผูกติดกัน และเพิ่มความสามารถในการ deploy/scale/ดูแลแยกส่วน โดยควบคุมความสอดคล้องของข้อมูลและ API

เป้าหมาย SMART

- ภายใน 2 ไตรมาส: availability ≥ 99.9%, MTTR ≤ 30 นาที
- Deployment frequency เพิ่ม ≥ 50% และ lead time ลด ≥ 30%
- API contract compliance 100%; data ownership violation = 0
- Critical service มี observability ครบ ≥ 95%

สาระสำคัญตามหัวข้อย่อย

หัวข้อย่อย	สาระสำคัญ
บริการย่อย (Services)	แยกตามความสามารถธุรกิจ เช่น Order, Payment, Inventory, Notification
ขอบเขตแต่ละบริการ	ต้องชัดว่า service ใดรับผิดชอบอะไร และมี ownership ของข้อมูล/กระบวนการ
API	ใช้ contract ชัดเจน เช่น REST/GraphQL/gRPC มี versioning และ schema validation
Data Ownership	แต่ละ service เป็นเจ้าของข้อมูลตน ไม่เขียนข้าม service โดยตรง
การสื่อสารระหว่างบริการ	sync ผ่าน API หรือ async ผ่าน event/message broker; ระวัง coupling และ retry/idempotency
Deployment	deploy อิสระด้วย CI/CD, container/orchestration, rollback ได้

หัวข้อย่อย	สาระสำคัญ
Scaling	scale ตาม service ที่เป็นคอขวด เช่น horizontal scaling
Monitoring	ใช้ logs, metrics, traces, alerting และ health check
เทียบ Monolithic	ข้อดี: อิสระ/scale แยก/ทีมแยก; ข้อเสีย: ซับซ้อนด้าน data consistency, network, ops
ลดความเสี่ยง	เริ่มจาก modular monolith หรือ strangler pattern, มี contract test, observability, circuit breaker

ข้อดี

- แต่ละบริการมีขอบเขตชัด ทีมทำงานอิสระได้
- Deploy/scale แยกส่วน ลดผลกระทบวงกว้าง
- เลือกเทคโนโลยีตามความเหมาะสมของบริการได้
- เหมาะกับระบบที่เติบโตและต้องการความคล่องตัว

ข้อเสีย

- ซับซ้อนกว่า Monolithic ทั้ง deployment, monitoring, network
- ความสอดคล้องข้อมูลข้ามบริการยากขึ้น
- มี latency/network failure และต้องจัดการ retry/idempotency
- ต้นทุน ops และทักษะทีมสูง

ข้อควรระวัง / แนวทางลดความเสี่ยง

- อย่าแบ่ง service ตามชั้นเทคนิค แต่แบ่งตาม business capability
- กำหนด data ownership และกฎห้ามเขียนข้ามฐานข้อมูล
- ทำ API contract/versioning และ contract test
- ใช้ async event เมื่อเหมาะสม พร้อม outbox/idempotency
- มี observability, circuit breaker, timeout, retry, bulkhead
- เริ่มจากขอบเขตเล็กหรือ modular monolith ก่อนแยก

ตัวอย่าง

ระบบ e-commerce: แยก Order Service, Payment Service, Inventory Service, Notification Service โดย Order สื่อสารกับ Payment/Inventory ผ่าน API หรือ event และแต่ละ service มีฐานข้อมูลของตน

ตัวชี้วัดเรื่องนี้ (KPI)

- Availability / uptime ของบริการหลัก
- Latency p95/p99 และ error rate
- Deployment frequency และ lead time
- MTTR และจำนวน incident จาก cross-service failure
- API contract compliance และจำนวน breaking change
- Data ownership violation = 0
- Scaling efficiency/ต้นทุนต่อ transaction

การประยุกต์ใช้

ใช้กับระบบที่ต้องการ scale แยกส่วน ทีมหลายทีม หรือระบบที่เติบโตเร็ว โดยเริ่มจากขอบเขตธุรกิจชัดและมีความพร้อมด้าน DevOps/Observability

---

2. Domain-Driven Design (DDD)

เรื่อง

Domain-Driven Design — ออกแบบซอฟต์แวร์โดยยึดโดเมนธุรกิจเป็นศูนย์กลาง ใช้ภาษาเดียวกันและแบ่ง bounded context ให้ชัด

วัตถุประสงค์

ทำให้โมเดลซอฟต์แวร์สะท้อนธุรกิจจริง ลดความกำกวมในการสื่อสาร และควบคุมขอบเขตของโดเมนไม่ให้ปนกัน

เป้าหมาย SMART

- ภายใน 1–2 ไตรมาส: มี glossary/ubiquitous language ครอบคลุม ≥ 90% ของคำสำคัญ
- Bounded context และ context map ผ่านความเห็นชอบ 100%
- Aggregate invariants มี test ครอบคลุม ≥ 90%
- Domain event delivery ≥ 99.9% สำหรับ event สำคัญ
- ลด cross-context coupling ที่ไม่จำเป็น ≥ 30%

สาระสำคัญตามหัวข้อย่อย

หัวข้อย่อย	สาระสำคัญ
Domain / Subdomains	แยก core domain, supporting subdomain, generic subdomain เพื่อจัดลำดับความสำคัญ
Bounded Contexts	ขอบเขตที่โมเดลและภาษาเดียวกันใช้ได้ชัดเจน
Ubiquitous Language	ภาษากลางระหว่างธุรกิจและทีมเทคนิค ใช้ในเอกสาร โค้ด และการสื่อสาร
Entities	วัตถุที่มี identity ต่อเนื่อง เช่น Order, Customer
Value Objects	วัตถุที่ไม่มี identity ใช้ค่าเป็นหลัก เช่น Money, Address
Aggregates	กลุ่ม entity/value object ที่รักษา invariant ผ่าน aggregate root
Repositories	abstraction สำหรับเข้าถึง aggregate โดยไม่ผูกกับฐานข้อมูล
Domain Events	เหตุการณ์สำคัญในโดเมน เช่น OrderPlaced, PaymentCaptured
Context Mapping	แผนที่ความสัมพันธ์ระหว่าง bounded context เช่น shared kernel, customer-supplier, anti-corruption layer
โครงสร้างโค้ดสะท้อนโดเมน	จัด package ตาม domain/context ไม่ใช่ตาม layer เทคนิคเพียงอย่างเดียว

ข้อดี

- โมเดลตรงธุรกิจ ลด miscommunication
- ขอบเขตชัด ลดการปน logic ข้ามโดเมน
- Aggregate/repository/domain event ช่วยรักษากฎธุรกิจ
- รองรับระบบซับซ้อนและทีมหลายทีม

ข้อเสีย

- ต้องเข้าถึงผู้เชี่ยวชาญธุรกิจและใช้เวลาเรียนรู้
- เสี่ยง over-modeling ในระบบเล็ก
- ต้องมีวินัยเรื่อง ubiquitous language และ context boundary
- Context mapping/event อาจเพิ่มความซับซ้อน

ข้อควรระวัง

- อย่าใช้ DDD ทุกระบบ โดยเฉพาะระบบ CRUD ง่าย ๆ
- ระวัง aggregate ใหญ่เกินไปและ transaction ข้าม aggregate
- ใช้ anti-corruption layer เมื่อเชื่อมระบบภายนอก
- ทบทวน glossary และ context map เป็นระยะ

ตัวอย่าง

ระบบ e-commerce: Bounded Contexts ได้แก่ Sales, Inventory, Billing, Shipping; ใน Sales มี Aggregate `Order`, Value Object `Money`, Domain Event `OrderPlaced`; เชื่อม Billing ผ่าน context mapping และ anti-corruption layer

ตัวชี้วัดเรื่องนี้ (KPI)

- % คำสำคัญที่มี ubiquitous language glossary
- % bounded context/context map ที่ได้รับอนุมัติ
- % aggregate invariant ที่มี automated test
- จำนวน cross-context dependency ที่ไม่ผ่าน ACL
- Domain event success rate/latency
- % โมดูลที่จัดโครงสร้างตามโดเมน

การประยุกต์ใช้

ใช้กับระบบธุรกิจซับซ้อน เช่น ERP/CRM, e-commerce, ระบบการเงิน และระบบที่มีหลายทีม/หลายโดเมน

---

3. Clean Architecture

เรื่อง

Clean Architecture — แยกชั้นโค้ดให้ business logic เป็นศูนย์กลางและไม่ผูกกับ database/framework/UI

วัตถุประสงค์

ทำให้ business logic ทดสอบได้ เปลี่ยนเทคโนโลยีได้ง่าย และลดผลกระทบจากการเปลี่ยนแปลงภายนอก

เป้าหมาย SMART

- ภายใน 2 ไตรมาส: dependency rule violation ใน review = 0 สำหรับโมดูลหลัก
- Use case test coverage ≥ 80%
- Change failure rate ลด ≥ 20%; lead time ลด ≥ 30%
- POC เปลี่ยน adapter/เทคโนโลยีได้โดยไม่แก้ use case สำเร็จ ≥ 1 กรณี

ชั้นสถาปัตยกรรม

ชั้น	หน้าที่
1. Entities	กฎธุรกิจหลัก/domain model ที่ไม่ขึ้นกับเทคโนโลยี
2. Use Cases	กระบวนการธุรกิจ/application logic ที่ orchestrate entities
3. Interface Adapters	แปลงข้อมูลระหว่าง use case กับภายนอก เช่น controller, presenter, repository adapter
4. Frameworks & Drivers	เครื่องมือภายนอก เช่น web framework, database, UI, message broker

ข้อกำหนดสำคัญ

ข้อกำหนด	แนวปฏิบัติ
Dependency Rule	ชั้นในต้องไม่รู้จักชั้นนอก; dependency ชี้เข้าด้านในเท่านั้น
แยก Business Logic	Entities/Use Cases ไม่ผูกกับ database/framework/UI
วิธีทดสอบ	Unit test entities/use cases ด้วย mock repository; integration test adapter; contract test
เปลี่ยนเทคโนโลยี	เปลี่ยน adapter/framework โดยไม่แก้ use case; ใช้ interface/DI
ข้อดี	ทดสอบง่าย เปลี่ยน tech ง่าย ลด coupling
ข้อเสีย	มี abstraction/ไฟล์มากขึ้น อาจ over-engineer ในระบบเล็ก

ตัวอย่าง

Use Case `CreateOrder` เรียก `OrderRepository` interface; adapter ใช้ PostgreSQL หรือ in-memory สำหรับ test; controller/framework อยู่วงนอกและไม่ปน business logic

ตัวชี้วัดเรื่องนี้ (KPI)

- จำนวน dependency rule violation
- Use case/entity test coverage
- Change failure rate หลังเปลี่ยน adapter/framework
- Lead time ในการเพิ่ม use case ใหม่
- % use case ที่ไม่ import framework/database โดยตรง

การประยุกต์ใช้

ใช้กับระบบที่มี business logic ซับซ้อน ต้องการทดสอบง่าย หรือคาดว่าจะเปลี่ยน database/framework/UI ในอนาคต

4. บริหารความเสี่ยงโครงการ

เรื่อง

การบริหารความเสี่ยงโครงการ — ระบุ ประเมิน จัดลำดับ และกำหนดมาตรการ/ผู้รับผิดชอบ/แผนสำรอง

กระบวนการ

1. ระบุความเสี่ยง
2. ประเมินโอกาสเกิด (1–5)
3. ประเมินผลกระทบ (1–5)
4. จัดลำดับความเสี่ยง (ระดับ = โอกาส × ผลกระทบ)
5. มาตรการ: ลด/ป้องกัน/โอน/ยอมรับ
6. ผู้รับผิดชอบ
7. KPI ติดตาม
8. แผนสำรอง

ตาราง Risk Register (ตัวอย่าง)

ความเสี่ยง	โอกาส	ผลกระทบ	ระดับ	มาตรการ	ผู้รับผิดชอบ	KPI ติดตาม	แผนสำรอง
Requirement เปลี่ยนบ่อย	4	4	สูง	ลด: baseline + change control + prototype	SA/PM	จำนวน change request หลัง baseline	ใช้ buffer/phase gate
API breaking change	3	5	สูง	ป้องกัน: versioning + contract test	Tech Lead	% contract test ผ่าน	rollback/feature flag
Data inconsistency ข้าม service	3	5	สูง	ลด: outbox/idempotency/saga	Architect	จำนวน data mismatch incident	reconcile job
ทีมขาดทักษะเทคโนโลยีใหม่	3	3	กลาง	โอน/ลด: training + mentor	PM	% ทีมผ่าน training	ใช้ vendor/ที่ปรึกษา
Vendor lock-in	2	4	กลาง	ลด: abstraction/POC ทางเลือก	Architect	% dependency ผ่าน abstraction	แผนย้ายทางเลือก

KPI ระดับโครงการ

- % ความเสี่ยงสูงที่มีมาตรการและเจ้าภาพครบ
- จำนวนความเสี่ยงที่กลายเป็น issue
- % มาตรการเสร็จตามกำหนด

- % แผนสำรองที่ทดสอบแล้ว

การประยุกต์ใช้

ใช้ตั้งแต่เริ่มโครงการและทบทวนทุกรอบ phase gate หรือ sprint review

---

5. วางแผน POC (Proof of Concept)

เรื่อง

แผน POC สำหรับแนวคิด/เทคโนโลยี — พิสูจน์ความเป็นไปได้ก่อนลงทุนจริง

ลำดับ	หัวข้อ	สิ่งที่ต้องกำหนด
1	คำถามที่ต้องพิสูจน์	เช่น เทคโนโลยีนี้รองรับ throughput/latency ที่ต้องการได้หรือไม่
2	สมมติฐาน	เช่น event-driven ช่วยลด coupling และรองรับ scaling ได้
3	ขอบเขต	ทำเฉพาะ scenario สำคัญ ไม่ทำทั้งระบบ
4	ทรัพยากร	คน/เครื่องมือ/สภาพแวดล้อม/งบประมาณ
5	ระยะเวลา	กำหนด timebox เช่น 2–4 สัปดาห์
6	ตัวชี้วัดความสำเร็จ	latency, throughput, cost, ความยากในการ deploy
7	เกณฑ์ Go / No-Go	ผ่านเกณฑ์ขั้นต่ำทั้งหมดหรือไม่
8	สรุปผลและข้อเสนอแนะ	ผลทดลอง ความเสี่ยง ต้นทุน และข้อเสนอไปต่อ/หยุด

ตัวอย่าง POC

พิสูจน์ว่า Kafka + microservices รองรับ Order/Payment event ที่ 1,000 msg/s, latency p95 ≤ 200 ms, และทีม deploy ได้ภายใน 1 วัน โดยมีเกณฑ์ Go/No-Go ชัดเจน

KPI POC

- % คำถามที่พิสูจน์ได้ครบ
- % เกณฑ์ Go/No-Go ที่ผ่าน
- เวลาที่ใช้เทียบ timebox
- ต้นทุน POC เทียบงบ
- จำนวนข้อค้นพบ/ความเสี่ยงที่ระบุได้

---

6. บริหารโครงการ

เรื่อง

การบริหารโครงการ — ควบคุม scope, stakeholder, WBS, เวลา, งบ, ทรัพยากร, ความเสี่ยง, การสื่อสาร, คุณภาพ และการปิดโครงการด้วยหลัก SMART

องค์ประกอบ	แนวปฏิบัติ	KPI/หลักฐาน
Scope	กำหนด in/out scope และ change control	% scope change หลัง baseline
Stakeholders	วิเคราะห์ผู้มีส่วนได้ส่วนเสียและแผนสื่อสาร	% stakeholder engagement/ความพึงพอใจ
WBS	แตกงานเป็น deliverables ที่วัดได้	% งานตาม WBS เสร็จตามกำหนด
Timeline	milestones + critical path	% ส่งมอบตาม milestone
Budget	ประมาณการและติดตาม variance	% งบใช้เทียบแผน
Resources	คน/ทักษะ/เครื่องมือ	% resource utilization/ช่องว่างทักษะ
Risk	Risk Register + แผนสำรอง	% ความเสี่ยงสูงมีมาตรการ
Communication	ช่องทาง/ความถี่/ผู้รับ	% การสื่อสารตรงรอบ
Quality	เกณฑ์ยอมรับ/test/UAT	% UAT ผ่าน/defect หลุด production
KPI	SMART และทบทวนตามรอบ	% KPI หลักบรรลุเป้า
การปิดโครงการ	ส่งมอบ/ตรวจรับ/สรุปบทเรียน	% ปิดโครงการครบเอกสาร; lessons learned

หลัก SMART และบทเรียน

- ตั้งเป้า Specific, Measurable, Achievable, Relevant, Time-bound
- ทบทวนบทเรียนทุก phase: อะไรดี/อะไรควรปรับ/จะทำ differently อย่างไร
- เก็บ lessons learned เป็น action item พร้อมเจ้าภาพและกำหนดเวลา

ตัวอย่าง

โครงการ ERP rollout: กำหนด scope ชัด แยก WBS ตามโมดูล ตั้ง milestone UAT, ติดตามงบรายเดือน, มี Risk Register และปิดโครงการด้วย lessons learned

---

สรุปภาพรวม

> ** Microservices | แยกบริการตามธุรกิจ มี API/data ownership/observability และควบคุมความซับซ้อน ** DDD | ยึดโดเมน ภาษาเดียวกัน และ bounded context เพื่อลดความกำกวม ** Clean Architecture | Business logic อยู่ศูนย์กลาง dependency ชี้เข้าใน เปลี่ยนเทคโนโลยีได้ง่าย ** บริหารความเสี่ยง | ระบุ–ประเมิน–จัดลำดับ–มาตรการ–เจ้าภาพ–KPI–แผนสำรอง ** POC | พิสูจน์สมมติฐานด้วยขอบเขต/timebox/เกณฑ์ Go–No-Go ชัด ** บริหารโครงการ | ควบคุม scope/stakeholder/WBS/เวลา/งบ/คุณภาพ ด้วย SMART และบทเรียน